Task-3:- Documentation(Automate Deployment + Database Backup)

1. Project Overview

This project involved deploying a 2-tier Node.js and MySQL application on AWS EC2. The objective was to automate the continuous deployment process and ensure data persistence through scheduled database backups to AWS S3.

Tech Stack:

- **Backend:** Node.js, Express.js (hosted on EC2)
- **Database:** MySQL (hosted on a separate EC2)
- **Process Management:** PM2
- **Storage:** AWS S3
- **Automation:** Bash Scripting & Cron jobs

---

2. Infrastructure Setup

The application is hosted across two separate EC2 instances to ensure security and resource isolation.

- **Instance A (App Server):** Runs the Node.js backend.
- **Instance B (DB Server):** Runs the MySQL database.
- **S3 Bucket:** Used as a remote repository for database snapshots.

---

3. Architecture and Infrastructure

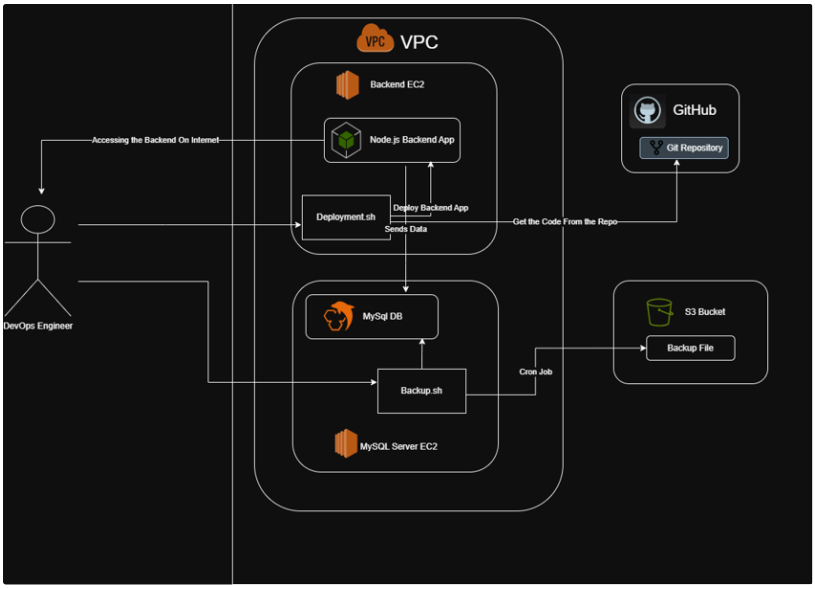
3.1 Infrastructure Overview

Core Components

- Linux Server (Ubuntu)
- Node.js Runtime
- PM2 Process Manager
- MySQL Server
- AWS CLI
- Amazon S3 Bucket
- Cron Scheduler
- GitHub Repository

Application Architecture and Data Flow

1. Administrator runs deployment script
2. Application is cloned and started via PM2
3. Backup script runs hourly via cron
4. mysqldump creates database dump
5. Backup uploaded to Amazon S3
6. Logs recorded locally



3.2 Environments

Environment	Purpose
Development	Local testing
Staging	Deployment validation
Production	Live application and backups

3.3 Tools and Technologies

Tool	Purpose
Node.js	Application runtime
PM2	Process management
MySQL	Database
mysqldump	Database backup
AWS CLI	Cloud interaction
Amazon S3	Backup storage
GitHub	Source repository
Cron	Job scheduling
Bash	Automation scripting

3.4 User Request Flow

1. User accesses application via server IP
2. PM2-managed Node.js app handles request
3. Database operations performed on MySQL
4. Backup process runs independently via cron
5. Backup stored securely in S3

4. Automation Scripts

A. Deployment Script (`deploy.sh`)

This script automates the process of updating the application from the GitHub repository and restarting the service.

```
#!/bin/bash
# Configuration
REPO_PATH="/home/ubuntu/nodejs-express-mysql"
LOG_FILE="/home/ubuntu/deployment.log"

echo "--- Deployment started at $(date) ---" >> $LOG_FILE
cd $REPO_PATH || exit

# Pull latest code
git pull origin master >> $LOG_FILE 2>&1

# Install dependencies
npm install >> $LOG_FILE 2>&1

# Restart application with PM2
pm2 restart server.js >> $LOG_FILE 2>&1

echo "Deployment finished successfully at $(date)" >> $LOG_FILE
echo "-------------------------------------" >> $LOG_FILE

```

B. Database Backup Script (`db_backup.sh`)

This script performs a logical backup of the MySQL database and uploads the encrypted/compressed file to an S3 bucket.

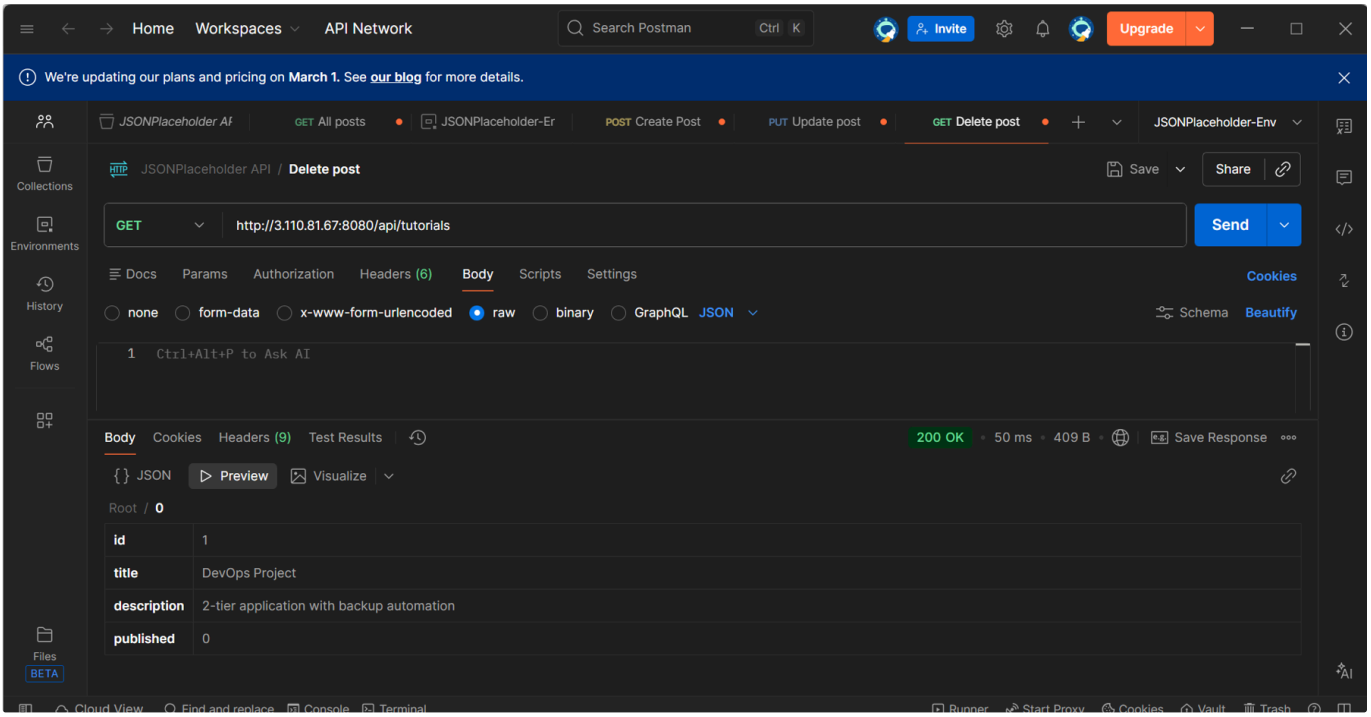
```
#!/bin/bash
# Configuration
DB_NAME="testdb"
BACKUP_DIR="/home/ubuntu/backups"
S3_BUCKET="s3://your-backup-bucket-name"
TIMESTAMP=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_FILE="$BACKUP_DIR/db_backup_$TIMESTAMP.sql"

# Create backup directory if not exists
mkdir -p $BACKUP_DIR

# Take MySQL Dump
mysqldump -u root -p'password' $DB_NAME > $BACKUP_FILE

# Upload to S3
aws s3 cp $BACKUP_FILE $S3_BUCKET/

# Optional: Remove local file to save space
rm $BACKUP_FILE

```

---

5. Implementation Proof (Screenshots)

5.1. Application Deployment & PM2 Management

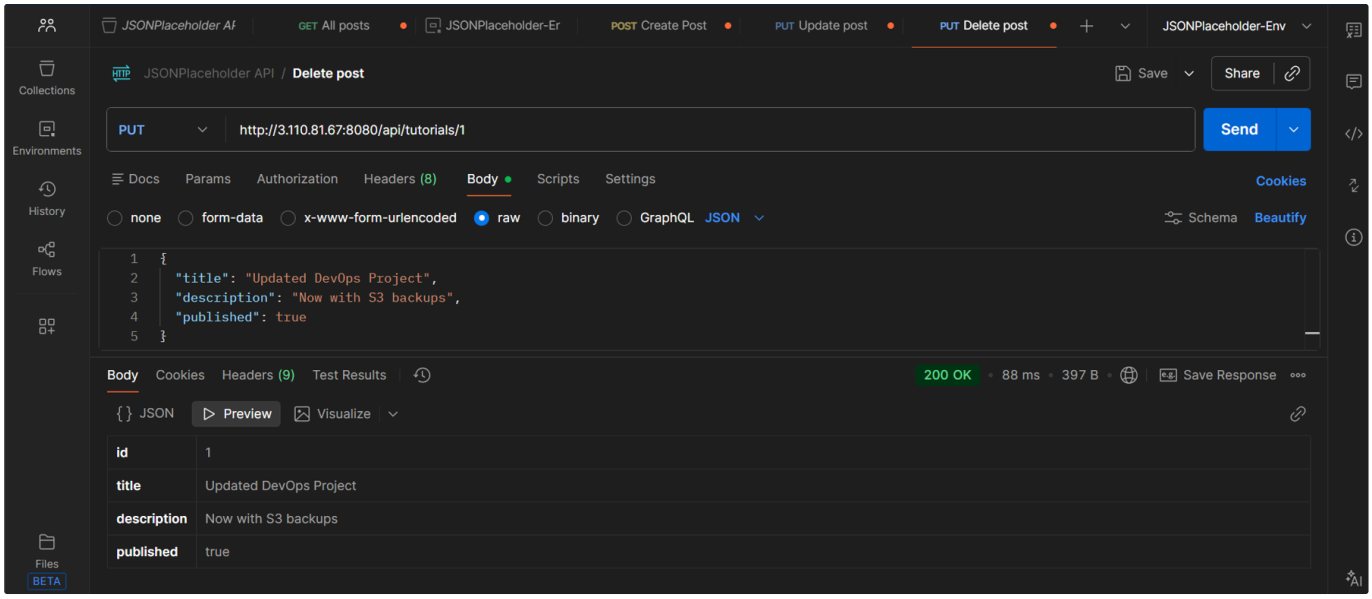
The following screenshots show the backend server running and the PM2 process manager maintaining the application uptime.



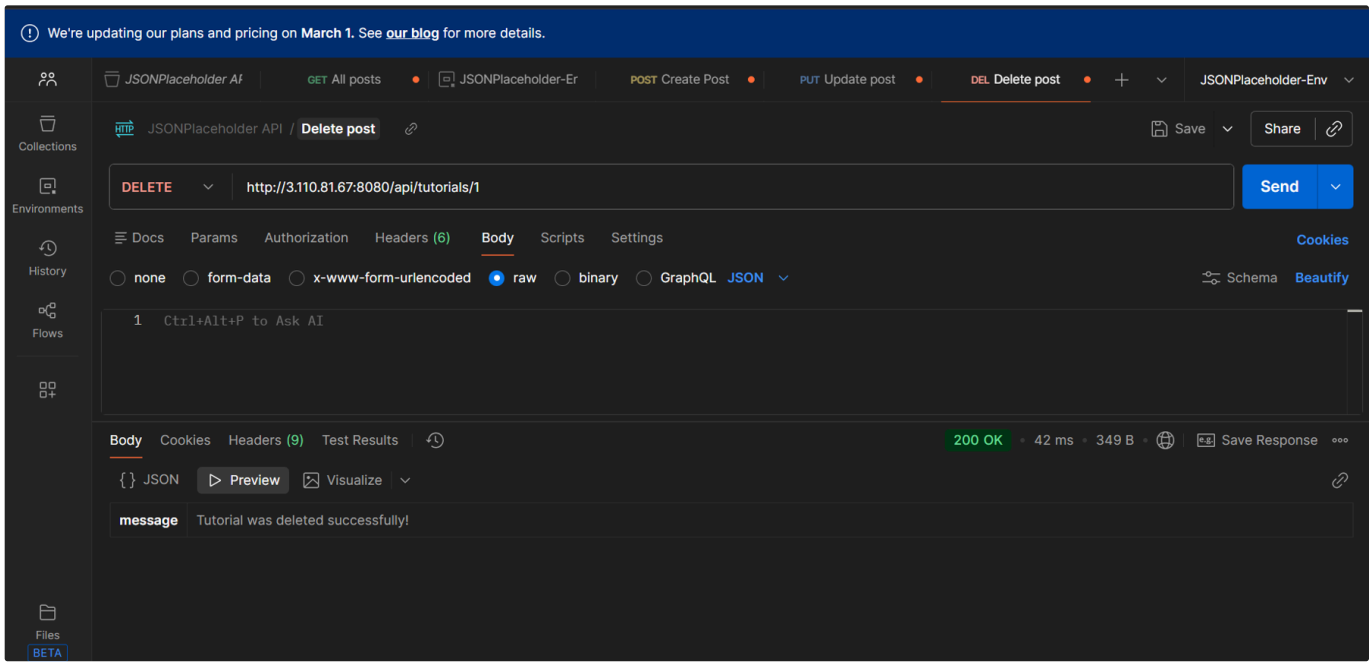
5.2 Database Verification

Verified that tutorial records are being stored and updated in MySQL database.

```
ubuntu@ip-172-31-43-153:~$ sudo mysql
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.45-0ubuntu0.24.04.1 (Ubuntu)

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE testdb
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> select * from tutorials
    -> ;
+----+----------------+-------------------------------------------+-----------+
| id | title          | description                               | published |
+----+----------------+-------------------------------------------+-----------+
|  1 | DevOps Project | 2-tier application with backup automation |         0 |
+----+----------------+-------------------------------------------+-----------+
1 row in set (0.00 sec)

mysql>
```




```
mysql> select * from tutorials;
+----+-------------------------+---------------------+-----------+
| id | title                   | description         | published |
+----+-------------------------+---------------------+-----------+
|  1 | Updated DevOps Project  | Now with S3 backups |         1 |
+----+-------------------------+---------------------+-----------+
1 row in set (0.00 sec)

mysql> |
```



```
mysql> select * from tutorials;
Empty set (0.00 sec)

mysql>
```

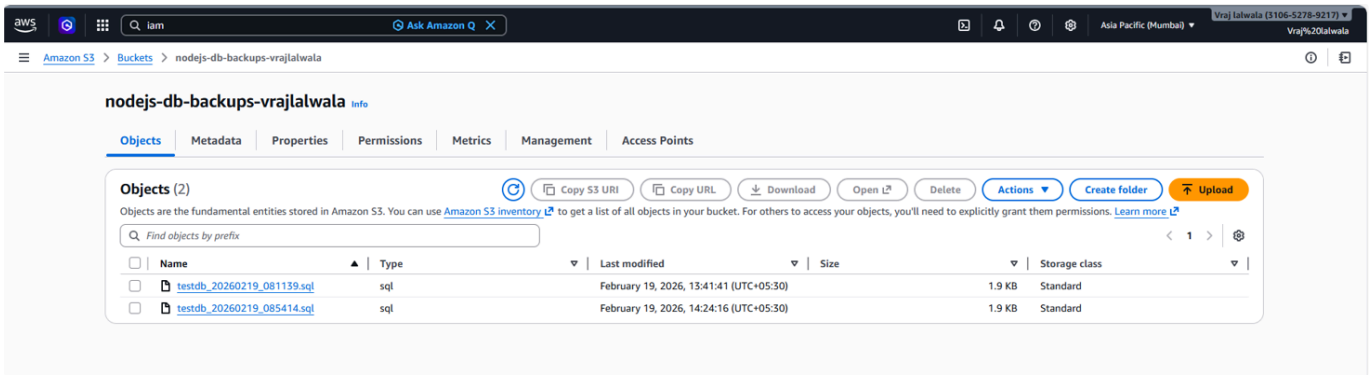
5.3. Database Backup Automation

The backup script was scheduled using `crontab` to run every hour.

```
ubuntu@ip-172-31-43-153: ~    ubuntu@ip-172-31-45-175: ~
  GNU nano 7.2                    /tmp/crontab.WkwuIa/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 * * * * /home/ubuntu/db_backup.sh >> /home/ubuntu/backup.log 2>&1

                                  [ Read 24 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location  M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

5.4. AWS S3 Verification

The ultimate proof of the backup automation is the presence of timestamped `.sql` files in the S3 bucket.



---

6. Final Architecture Flow

User → Public IP (Port 8080)
↓
Node.js Application (PM2 Managed)
↓
MySQL Database (Private EC2)
↓
Backup Script (Hourly via Cron)
↓
Amazon S3 (Backup Storage)

---

7. Conclusion

The task was successfully completed. The application is now capable of self-updating via the `deploy.sh` script, and the database is protected against data loss with an automated, off-site backup strategy.